PROGRAMA DE CAPACITACIÓN

Desarrollando con IA

Glosario del curso

Los términos clave de IA generativa, agentes, desarrollo moderno y gobernanza, explicados en simple para devs que están partiendo.

425	16	U1–U10
términos	temas	todo el curso

Material de consulta del programa, con práctica en Claude y Claude Code

Claude | ANTHROP\C

TEMA 01

01 Inteligencia artificial generativa

29 términos

Cómo piensan, qué hacen y dónde fallan los modelos

Agente de IA *AI agent*

Sistema que no solo responde: planifica, usa herramientas (leer archivos, ejecutar comandos, editar código) y verifica el resultado. Repite el ciclo reunir contexto, actuar y verificar hasta terminar la tarea. Requiere supervisión humana.

AI Fluency *AI Fluency*

Marco educativo de Anthropic sobre cómo colaborar con la IA de forma eficaz, ética y segura. Su idea central son las "4D": Delegación, Descripción, Discernimiento y Diligencia.

Alucinación *Hallucination*

Cuando el modelo inventa algo falso con total seguridad: una función que no existe, una cifra o una cita inexistente. No miente a propósito, solo completa con lo más probable. Por eso todo lo que genera se revisa y se prueba.

Asistente *Assistant*

Herramienta de IA que responde tus preguntas y genera contenido, pero no actúa por sí sola sobre tu proyecto. Tú copias, pegas y ejecutas. Es la diferencia clave con un agente, que sí puede actuar.

Benchmark *Benchmark*

Prueba estandarizada para comparar modelos en tareas como programación o razonamiento. Sirve de referencia, pero un buen puntaje no garantiza que el modelo rinda igual en tu proyecto real.

Codex *OpenAI Codex*

Agente de programación de OpenAI, comparable a Claude Code. Lee los archivos de instrucciones de un proyecto como AGENTS.md. Cada herramienta tiene detalles propios, pero los conceptos de fondo se transfieren de una a otra.

Delegación *Delegation*

Primera "D" del marco 4D: decidir qué tareas conviene entregar a la IA, cuáles delegar con revisión y cuáles no delegar nunca. Por ejemplo, elegir la pasarela de pagos de una empresa no se delega.

Descripción *Description*

Segunda "D": explicar con claridad qué necesitas, con qué restricciones y cómo se verificará el resultado. Un pedido vago produce respuestas vagas; uno con objetivo y criterios produce respuestas útiles.

Diligencia *Diligence*

Cuarta "D": responsabilidad sobre lo que se entrega. Implica revisar, cuidar los datos, transparentar que se usó IA y asumir las consecuencias como propias. "La IA lo hizo" nunca es una excusa.

Discernimiento *Discernment*

Tercera "D": evaluar con criterio lo que la IA entrega. Se lee, se ejecuta, se pregunta "¿por qué lo hiciste así?" y se buscan errores o supuestos inventados antes de aceptar nada.

Eval *Evaluation*

Conjunto de pruebas que mide qué tan bien responde un sistema con IA a casos concretos. Se parece a los tests de software, pero revisa la calidad de las respuestas del modelo en vez de resultados exactos.

GitHub Copilot *GitHub Copilot*

Asistente de programación con IA de GitHub y Microsoft que sugiere y genera código dentro del editor. Es uno de los "otros asistentes" con los que se compara Claude Code, y funciona con ideas similares.

Ingeniería asistida por IA *AI-assisted engineering*

Forma profesional de programar con IA: se define qué construir, se revisa cada cambio, se prueba y se responde por el resultado. La IA acelera el trabajo, pero la responsabilidad sigue siendo del desarrollador.

Ingeniería de contexto *Context engineering*

Decidir qué información tiene delante el modelo o agente: instrucciones del proyecto, archivos relevantes, resultados previos. Ya no se trata solo de "pedir bien", sino de darle la información justa, ni de más ni de menos.

Ingeniería de prompts *Prompt engineering*

Práctica de redactar pedidos claros para obtener mejores respuestas: indicar objetivo, contexto, formato esperado y ejemplos. Es el primer escalón; hoy se complementa con la ingeniería de contexto.

LLM *Large Language Model*

Modelo de lenguaje grande: un programa entrenado con enormes cantidades de texto que predice qué viene a continuación en una frase. Con eso puede conversar, resumir, traducir y escribir código. No "sabe" cosas como una persona: genera la respuesta más probable según lo que aprendió.

Marco 4D *4D framework*

Cuatro habilidades para trabajar con IA: Delegación (decidir qué tarea entregarle), Descripción (explicarla bien), Discernimiento (evaluar críticamente lo que devuelve) y Diligencia (asumir la responsabilidad del resultado final).

Modelo *Model*

El "cerebro" entrenado que recibe tu pedido y genera la respuesta. Cada proveedor ofrece varios modelos: unos más rápidos y baratos, otros más potentes y lentos. Elegir el modelo según la tarea es parte del oficio.

Modelo abierto *Open-weight model*

Modelo cuyos archivos puedes descargar y ejecutar por tu cuenta, en vez de usarlo solo a través de la nube de un proveedor. Ofrece más control y privacidad, pero requiere hardware propio y suele rendir menos que los modelos más grandes.

Modelo local *Local model*

Modelo que corre en tu propio computador o servidor, sin enviar tu código a internet. Es útil cuando los datos son muy sensibles o no hay conexión. Su calidad depende del modelo y de la potencia de tu equipo.

No determinismo *Non-determinism*

Que el mismo pedido no siempre produzca la misma respuesta. Los modelos tienen un componente de azar, así que dos ejecuciones pueden diferir. Para trabajo serio se verifica el resultado en vez de confiar en que "salió bien la otra vez".

Ollama *Ollama*

Herramienta gratuita para descargar y ejecutar modelos abiertos en tu computador con unos pocos comandos. Es una forma sencilla de probar modelos locales y sirve como plan B cuando falla la red o un proveedor.

Prompt *Prompt*

El texto que le entregas al modelo para que haga algo: una pregunta, una instrucción, un ejemplo o un archivo. La calidad de la respuesta depende mucho de lo claro y completo que sea el prompt.

Prompt del sistema *System prompt*

Instrucciones de fondo que definen cómo debe comportarse el modelo antes de que escribas tu primer mensaje: su rol, tono y reglas. Normalmente las fija la herramienta o el equipo, no el usuario final.

Temperatura *Temperature*

Parámetro que regula cuánto azar usa el modelo al elegir la siguiente palabra. Temperatura baja: respuestas más predecibles y consistentes. Temperatura alta: respuestas más variadas y creativas, pero con más riesgo de errores.

Token *Token*

Trozo de texto que el modelo lee y escribe: puede ser una palabra corta, parte de una palabra o un signo. Los modelos no cuentan letras, cuentan tokens. El uso, el costo y el límite de memoria se miden en tokens.

Uso de herramientas *Tool use*

Capacidad del modelo de pedir que se ejecute una acción externa, como leer un archivo, correr un test o consultar una base de datos, y luego usar el resultado. Es lo que convierte a un modelo que conversa en un agente que trabaja.

Ventana de contexto *Context window*

Cantidad máxima de texto (en tokens) que el modelo puede tener presente a la vez: tus mensajes, sus respuestas y los archivos que leyó. Es como una mesa de trabajo de tamaño fijo: si se llena, lo antiguo se pierde o debe resumirse.

Vibe coding *Vibe coding*

Programar describiendo "la onda" a una IA y aceptando lo que salga sin leerlo ni probarlo a fondo. Sirve para prototipos desechables o ideas personales; es una mala práctica para software que irá a producción por los errores y fallos de seguridad que oculta.

TEMA 02

02 Claude y Claude Code

Las herramientas de Anthropic y su vocabulario

50 términos

/agents

Comando para crear y administrar subagentes: asistentes especializados con su propio contexto y herramientas limitadas.

Anthropic

Empresa de inteligencia artificial que crea los modelos Claude. También impulsó estándares abiertos que hoy usa toda la industria, como MCP y Agent Skills.

API *API*

Application Programming Interface: puerta por la que un programa pide cosas a otro. Usar la API de Claude significa conectar tu propio código a los modelos y pagar por lo que consumes, aparte de los planes de chat.

Aprobación manual *Manual approval*

Modo de permisos en que Claude Code pide tu autorización antes de cada edición o comando. Es el más seguro y el recomendado mientras aprendes.

Asiento *Seat*

Licencia asignada a una persona dentro de un plan de equipo o empresa. Cada asiento tiene su propia cuota de uso, así que una persona intensiva puede agotarla antes que otra.

Carpeta .claude/

Directorio del proyecto donde Claude Code guarda su configuración compartible: subagentes, Skills, settings.json y hooks. Si se versiona en el repositorio, todo el equipo trabaja con las mismas reglas.

Claude

Familia de modelos de IA y asistente de Anthropic. Se usa desde la web, la app de escritorio o el móvil, y desde herramientas de programación como Claude Code. Es la herramienta de referencia de este curso.

Claude Academy

Plataforma de cursos oficiales de Anthropic sobre Claude, Claude Code, agentes y buenas prácticas. Es una de las fuentes principales de este curso.

Claude Certified Developer – Foundations

Certificación de Anthropic para desarrolladores que usan Claude. El examen está abierto a personas de organizaciones del Claude Partner Network.

Claude Code

Agente de programación de Anthropic: lee tu proyecto, edita archivos, ejecuta comandos y tests, y propone cambios mientras tú supervisas. Funciona en terminal, en IDEs como los de JetBrains, en escritorio y en la web.

Claude Console

Portal de Anthropic para desarrolladores donde se crean claves de API, se controla el gasto y se gestiona el uso de la plataforma. No es lo mismo que la cuenta de chat.

Claude Desktop

Aplicación de Claude para computador. Además del chat, incluye una pestaña de código para trabajar con Claude Code, útil como plan B si falla la terminal.

Claude GitHub App

Aplicación que se instala en un repositorio de GitHub para que Claude pueda leer y responder en issues y pull requests. Es un paso previo para usar Claude dentro de GitHub Actions.

Claude in Chrome

Extensión del navegador Chrome que permite a Claude navegar, leer páginas y probar interfaces web por ti. Siempre conviene revisar qué permisos le das.

claude setup-token

Comando de Claude Code que genera el token de suscripción. Se usa para autenticar automatizaciones, como un workflow de GitHub Actions, con el asiento de quien lo ejecuta.

Claude Tag

Forma de usar Claude dentro de Slack: se menciona a @Claude en un hilo y trabaja con las herramientas y el contexto del equipo. Lo configura un administrador de la organización.

Clave de API *API key*

Contraseña larga que identifica tu programa ante un servicio y permite cobrarle el uso. Se trata como un secreto: nunca va en el código ni en el repositorio, solo en un lugar seguro como los secrets de GitHub.

/clear

Comando que borra el contexto de la sesión y empieza de cero. Úsalo al cambiar de tarea para que lo anterior no confunda al agente.

/code-review

Comando local que pide a Claude revisar tus cambios y listar hallazgos antes de abrir un pull request. Cuenta contra la cuota de tu asiento.

/compact

Comando que resume la conversación actual para liberar espacio en la ventana de contexto, conservando lo esencial. Úsalo cuando la sesión es larga pero aún necesitas su historia.

/config

Comando que abre la configuración de Claude Code, donde se ajustan opciones como la lectura de instrucciones del proyecto.

Configuración administrada *Managed settings*

Reglas definidas por la administración de una organización que se aplican a todos los usuarios y no se pueden saltar individualmente. Es el nivel más fuerte de una política ejecutable.

/context

Comando que muestra cuánto de la ventana de contexto está ocupando cada cosa: instrucciones, archivos, mensajes. Sirve para entender por qué el agente "olvida" o se vuelve lento.

/cost

Comando que muestra cuánto has consumido en la sesión actual. Ayuda a tomar conciencia del costo de cada tarea y a compararlo con el valor que entrega.

Cuota de uso *Usage limit*

Cantidad de uso que tu plan permite en un periodo. Cuando se agota, debes esperar o cambiar de plan. Un uso intensivo de agentes consume más cuota que un chat normal.

/feedback

Comando para enviar comentarios o reportes a Anthropic. Lo que se envía por esta vía se guarda por más tiempo que las sesiones normales, así que no incluyas datos sensibles.

Haiku, Sonnet y Opus

Los tres tamaños de modelos Claude. Haiku es el más rápido y económico, para tareas simples. Sonnet equilibra calidad y costo para el día a día. Opus es el más potente, pensado para tareas complejas. Los nombres y versiones cambian, por eso conviene revisar la documentación vigente.

/ide

Comando que conecta una sesión de Claude Code abierta en una terminal externa con tu IDE, por ejemplo uno de JetBrains. Es el plan B si el plugin falla.

/init

Comando que analiza el proyecto y genera un archivo de instrucciones inicial. Ojo: puede crear un CLAUDE.md, lo que podría anular un AGENTS.md existente; hay que revisar qué archivo produce.

/mcp

Comando que lista los servidores MCP conectados y las herramientas que ofrece cada uno. Sirve para confirmar que una conexión realmente funciona.

Memoria del proyecto *Project memory*

Instrucciones persistentes que el agente lee al empezar una sesión: comandos, estilo y reglas del proyecto. Se guarda en archivos como AGENTS.md o CLAUDE.md.

/model

Comando para cambiar de modelo durante la sesión, por ejemplo de Sonnet a Opus para una tarea difícil. Permite comparar resultados y ajustar costo y calidad.

Modo auto-aceptar *Auto-accept mode*

Modo en que Claude Code aplica las ediciones sin preguntar cada vez. Es más rápido, pero te exige revisar después el resultado y puede tocar archivos de configuración si no se limita.

Modo headless *Headless mode*

Ejecutar Claude Code sin interfaz interactiva, con el comando claude -p, para integrarlo en scripts o pipelines. Recibe una instrucción, hace el trabajo y devuelve el resultado.

OpenTelemetry *OpenTelemetry*

Estándar abierto para recolectar métricas, trazas y registros de un sistema. Permite que una organización mida el uso de sus herramientas con sus propios paneles.

Panel de analytics *Analytics dashboard*

Panel que muestra cómo se usa Claude Code en una organización: usuarios activos, sesiones, líneas aceptadas y tasa de aceptación. Es útil para ver adopción, pero no mide por sí solo el valor entregado.

/permissions

Comando que muestra y edita las reglas de permiso vigentes: qué puede hacer Claude sin preguntar, qué requiere aprobación y qué está prohibido.

Plan Enterprise *Enterprise plan*

Plan de Claude para organizaciones grandes, con controles de administración avanzados, opciones de retención de datos y analítica de uso.

Plan mode *Plan mode*

Modo de solo lectura de Claude Code: el agente explora, analiza y propone un plan, pero no modifica nada hasta que lo apruebes. Ideal para entender un código desconocido o revisar una estrategia antes de cambiarla.

Plan Team *Team plan*

Plan de Claude para equipos pequeños y medianos, con administración central y términos comerciales de uso de datos. Existen variantes (como Standard y Premium) con distintos límites de uso; conviene revisar la página oficial de planes.

/plugin

Comando para instalar, ver y administrar plugins y marketplaces dentro de Claude Code.

Projects *Projects*

Función de Claude en la web que agrupa archivos e instrucciones de un tema para que las conversaciones partan con ese contexto. Por ejemplo, subir el README de un sistema para consultarlo sin repetirlo cada vez.

Rewind *Rewind*

Función de Claude Code para volver atrás a un punto anterior de la sesión y deshacer cambios del agente. Junto con git, es una red de seguridad cuando algo sale mal.

/sandbox

Comando que activa el aislamiento a nivel de sistema operativo: limita a qué archivos y a qué red puede acceder Claude, incluso si algo sale mal.

/security-review

Comando que pide a Claude revisar los cambios pendientes buscando problemas de seguridad, como inyección SQL o secretos expuestos. No reemplaza una auditoría completa.

settings.json

Archivo de configuración de Claude Code. Define permisos (qué se permite o se prohíbe), hooks y otras opciones. Versionado en el repositorio, convierte una política escrita en una regla que se cumple sola.

Telemetría *Telemetry*

Datos técnicos de uso que una herramienta envía automáticamente al proveedor, como errores o métricas. Se puede limitar con variables de configuración si la política de datos lo exige.

Token de suscripción *Subscription token*

Credencial que permite usar tu suscripción de Claude desde una herramienta como GitHub Actions, sin API key. Consume la cuota de tu asiento y queda ligado a tu cuenta personal.

Transcripción *Transcript*

Registro de una sesión de Claude Code. Se guarda en tu computador en texto plano durante un tiempo, así que puede contener código y datos que escribiste: debe tratarse con cuidado.

Zero Data Retention *Zero Data Retention*

Acuerdo empresarial en que el proveedor no conserva los prompts ni las respuestas después de procesarlos. Reduce al mínimo la exposición de datos, pero se negocia a nivel de organización.

TEMA 03

03 Estándares y configuración de agentes

30 términos

Archivos y piezas que enseñan a un agente a trabajar contigo

AAIF *Agentic AI Foundation*

Fundación, parte de la Linux Foundation, que aloja estándares abiertos para agentes de IA como MCP y AGENTS.md. Busca que no dependan de un solo proveedor.

Agent Skills

Estándar abierto para enseñar a un agente un procedimiento reutilizable, empaquetado en una carpeta. El agente solo lo carga cuando la tarea lo necesita, en vez de tenerlo siempre en memoria.

AGENTS.md

Archivo de texto en la raíz del proyecto con las instrucciones para agentes de IA: cómo compilar, probar, qué estilo seguir y qué reglas respetar. Es un estándar abierto que leen varias herramientas de distintos proveedores.

allowed-tools

Campo opcional de una Skill que limita las herramientas que el agente puede usar mientras la ejecuta, por ejemplo solo leer y escribir en una carpeta. Es una forma de aplicar mínimo privilegio.

CLAUDE.md

Archivo de instrucciones propio de Claude Code, equivalente a AGENTS.md. Si existe uno, Claude Code puede ignorar el otro, por eso hay que cuidar que no queden ambos con contenido distinto.

Cliente MCP *MCP client*

Aplicación que se conecta a servidores MCP y usa lo que ofrecen. Claude Code actúa como cliente MCP.

code-modernization

Plugin oficial de Claude Code que apoya el análisis y la migración de sistemas legados. Se puede comparar su resultado con el análisis hecho a mano.

Código de salida 2 *Exit code 2*

En un hook de Claude Code, terminar con código 2 significa "bloquear". El mensaje de error se devuelve al agente para que corrija lo que hizo.

Description de una Skill

Texto breve que explica cuándo debe usarse la Skill. Es lo que el agente lee para decidir si la activa. Si una Skill no se activa sola, casi siempre el problema es una description poco clara.

Divulgación progresiva *Progressive disclosure*

Técnica de las Skills: el agente lee primero solo el nombre y la descripción, y carga el detalle únicamente si lo necesita. Así ahorra espacio en la ventana de contexto.

Estándar abierto *Open standard*

Especificación pública que cualquier empresa puede implementar. Reduce el "encierro" en un proveedor: lo que aprendes y los archivos que creas siguen sirviendo si cambias de herramienta.

Frontmatter *Frontmatter*

Bloque al inicio de un archivo Markdown, entre líneas de tres guiones, con datos en formato clave: valor. En un SKILL.md contiene el nombre y la descripción de la Skill.

Hook *Hook*

Comando que se ejecuta automáticamente cuando ocurre un evento del agente, como antes o después de editar un archivo. Permite aplicar reglas que no dependen de la memoria del desarrollador ni de la buena voluntad del agente.

Linux Foundation

Organización sin fines de lucro que cobija proyectos de software abierto de gran escala, como Linux y Kubernetes. Aloja también la fundación de estándares para agentes de IA.

Marketplace *Marketplace*

Catálogo de plugins. Uno interno es simplemente un repositorio de GitHub con un archivo marketplace.json, desde el cual los equipos instalan plugins aprobados.

marketplace.json

Archivo que lista los plugins disponibles en un marketplace, con su origen y descripción. Se agrega el repositorio con /plugin marketplace add.

Matcher *Matcher*

Filtro de un hook que indica a qué herramientas se aplica, por ejemplo solo a Edit y Write. Evita que el hook corra en cada acción innecesariamente.

MCP *Model Context Protocol*

Estándar abierto que conecta un agente de IA con herramientas y datos externos, como una base de datos o un servicio, mediante servidores con una interfaz común. Piénsalo como un "USB" para conectar capacidades a la IA.

.mcp.json

Archivo de configuración del proyecto que declara qué servidores MCP debe usar el agente y cómo conectarse a ellos. Se versiona para que todo el equipo comparta la misma conexión.

Playbook *Playbook*

Conjunto de buenas prácticas escritas de un equipo. Un "playbook como código" las empaqueta en un plugin instalable en vez de dejarlas en un documento que nadie lee.

Plugin *Plugin*

Paquete versionado que reúne Skills, subagentes, hooks y servidores MCP para instalarlos juntos con un comando. Es la forma de repartir buenas prácticas de un equipo a toda la organización.

plugin.json

Archivo de metadatos de un plugin, dentro de la carpeta .claude-plugin. Declara su nombre, versión y descripción.

PostToolUse

Evento de hook que se dispara después de que el agente usó una herramienta. Por ejemplo, ejecutar los tests tras cada edición y devolver el error al agente si fallan.

PreToolUse

Evento de hook que se dispara antes de que el agente use una herramienta. Sirve para bloquear acciones peligrosas, como leer un archivo .env, antes de que ocurran.

REVIEW.md

Archivo con reglas pensadas solo para revisar código: qué es importante, qué ignorar y cuántos comentarios menores permitir. Complementa a AGENTS.md y logra que la IA comente menos y mejor.

security-guidance

Plugin oficial de Claude Code que revisa el código mientras el agente lo escribe y avisa de patrones inseguros, como una consulta SQL concatenada. Avisa, pero no bloquea por sí solo.

Servidor MCP *MCP server*

Programa que expone herramientas o datos a un agente siguiendo el protocolo MCP. Por ejemplo, uno puede permitir consultar una base PostgreSQL en modo solo lectura. Conviene instalar solo servidores de origen confiable.

SKILL.md

Archivo principal de una Skill. Tiene arriba un encabezado con name y description, y abajo las instrucciones paso a paso. Puede venir acompañado de carpetas con referencias, plantillas y scripts.

Subagente *Subagent*

Asistente especializado que trabaja con su propio contexto y herramientas limitadas, y devuelve solo un resumen al agente principal. Sirve para explorar o revisar sin saturar la sesión principal.

Transporte MCP *MCP transport*

Forma en que cliente y servidor MCP se comunican: stdio (un programa local que se ejecuta en tu equipo) o HTTP (un servicio accesible por red).

TEMA 04

04 Metodología y proceso

Cómo se organiza y se construye el software en equipo

33 términos

Adopción *Adoption*

Grado en que un equipo realmente usa una herramienta o práctica de forma sostenida. Comprar licencias no es adoptar: hay que capacitar, acompañar y medir.

ADR / RDA *Architecture Decision Record*

Registro de decisión de arquitectura: documento corto que anota qué se decidió, por qué y qué alternativas se descartaron. Ayuda a que nadie repita la discusión meses después.

Bitácora *Log*

Registro cronológico de lo que se hizo, se decidió y se aprendió durante un proyecto. Sirve para retomar el trabajo sin perder el hilo.

Champion *Champion*

Persona de un equipo que impulsa la adopción de una nueva práctica, ayuda a sus compañeros y lleva el aprendizaje de vuelta al grupo.

Criterio de aceptación *Acceptance criterion*

Condición concreta que debe cumplirse para dar por terminada una historia. Debe poder responderse con sí o no y convertirse en una prueba automática.

Dado / Cuando / Entonces *Given / When / Then*

Formato para escribir criterios de aceptación: Dado un contexto, cuando ocurre una acción, entonces se espera un resultado. Hace el criterio claro y fácil de convertir en test.

Daily *Daily Scrum*

Reunión diaria breve del equipo: qué hice ayer, qué haré hoy y qué me bloquea. Trabajando solo, equivale a anotarlo cada mañana.

Definition of Done *Definition of Done*

Lista de condiciones que deben cumplirse para considerar algo "terminado": tests en verde, código revisado, documentación al día. Evita el clásico "casi listo".

Especificación *Specification*

Descripción precisa y verificable de lo que un software debe hacer. Una buena especificación permite que otra persona escriba las pruebas sin hacer preguntas.

Historia de usuario *User story*

Descripción breve de una necesidad desde el punto de vista de quien la tiene: "Como [rol], quiero [algo] para [beneficio]". Es una conversación inicial, no una especificación completa.

Incremento *Increment*

Parte del producto terminada, probada y funcionando al final de un sprint, que se suma a lo anterior y se puede mostrar.

intent.md

Documento corto que dice qué se quiere lograr, por qué y bajo qué restricciones. Es el punto de partida del flujo spec-driven, antes de escribir la especificación.

Kanban *Kanban*

Método para visualizar el trabajo en un tablero con columnas como "por hacer", "en curso" y "hecho". Limita cuántas tareas se hacen a la vez.

Multiplicador *Multiplier*

Desarrollador con experiencia que enseña y difunde buenas prácticas a otros. La adopción de una herramienta la empujan las personas, no las licencias.

MVP *Minimum Viable Product*

Producto mínimo viable: la versión más simple que ya entrega valor y permite aprender de usuarios reales antes de invertir más.

Nivel de madurez *Maturity level*

Escala para ubicar qué tan avanzada está una organización en una práctica. Sirve para decidir el siguiente paso en vez de querer saltar directo al final.

Piloto *Pilot*

Prueba pequeña y acotada de una nueva forma de trabajar antes de extenderla. Permite medir y ajustar con bajo riesgo.

plan.md

Plan de implementación que se genera a partir de la spec: enfoque, archivos afectados, riesgos y pasos. Se revisa y se aprueba antes de programar.

Product Backlog *Product Backlog*

Lista priorizada de todo lo que se quiere construir en un producto. El Product Owner la ordena según el valor para el cliente.

Product Owner *Product Owner*

Rol de Scrum responsable de decidir qué se construye y en qué orden, representando las necesidades del cliente o del negocio.

Retrospectiva *Retrospective*

Reunión donde el equipo analiza cómo trabajó y decide qué mejorar. En proyectos con IA, es buen momento para actualizar AGENTS.md con lo aprendido.

Scrum *Scrum*

Marco ágil para organizar el trabajo en periodos cortos llamados sprints, con roles y reuniones definidas. Dice qué se hace y en qué orden; no dice cómo programar cada cosa.

Scrum Master *Scrum Master*

Rol de Scrum que cuida que el equipo aplique bien el marco y remueve obstáculos. No es un jefe del equipo.

SDD *Spec-Driven Development*

Desarrollo guiado por especificaciones: primero se escribe con precisión qué debe hacer el software y cómo se comprobará, y recién después se genera el código. Evita que la IA tenga que inventar las reglas.

SDLC *Software Development Life Cycle*

Ciclo de vida del desarrollo de software: requisitos, diseño, construcción, pruebas, revisión, despliegue y mantención. Es el recorrido completo de una aplicación, desde la idea hasta su operación.

SDLC AI-native *AI-native SDLC*

Ciclo de desarrollo rediseñado alrededor de la IA: en vez de una línea con traspasos, es un ciclo donde cada etapa deja un artefacto versionado, como una spec o un plan, que la siguiente etapa lee.

Software factory *Software factory*

Empresa o área que desarrolla software para terceros de forma sistemática, con procesos y equipos repetibles.

spec.md

Especificación de una funcionalidad: historia de usuario, reglas de negocio, criterios de aceptación, ejemplos de entrada y salida, errores esperados y qué queda fuera. Es el documento más importante del flujo.

Sprint *Sprint*

Periodo corto y fijo, de una a cuatro semanas, en que el equipo construye un incremento de producto funcionando.

Sprint Review *Sprint Review*

Reunión al final del sprint donde se muestra lo que funciona a quien lo pidió y se recoge su opinión.

Stakeholder *Stakeholder*

Persona o grupo con interés en el resultado de un proyecto: clientes, usuarios, gerencia, soporte. Se les consulta porque el proyecto los afecta.

tasks.md

Lista de tareas pequeñas derivadas del plan, cada una con su forma de verificarse. Permite avanzar en incrementos revisables.

Trazabilidad *Traceability*

Poder seguir el hilo de un requisito a su implementación y a su prueba: cada criterio de aceptación se asocia a una tarea y a un test.

TEMA 05

05 Lenguajes de programación

Qué es cada uno y dónde brilla

23 términos

C# *C#*

Lenguaje moderno de tipado estático creado por Microsoft sobre la plataforma .NET. Se usa en APIs web, aplicaciones de escritorio, nube y videojuegos con Unity. Comparte muchas ideas con Delphi, por eso es un destino natural para migrarlo.

COBOL *COBOL*

Lenguaje de los años 50 aún presente en bancos, seguros y gobiernos, que procesa enormes volúmenes de transacciones. Quedan pocos especialistas, por eso su modernización es un caso de estudio frecuente.

Compilado e interpretado *Compiled / Interpreted*

Un lenguaje compilado se traduce a código de máquina antes de ejecutarse (Delphi, C#). Uno interpretado se traduce mientras corre (Python, PHP, JavaScript). Muchos mezclan ambas ideas.

Entorno virtual *Virtual environment (venv)*

Carpeta aislada donde se instalan las librerías de un proyecto Python sin afectar a otros proyectos ni al sistema. Evita conflictos entre versiones.

Java *Java*

Lenguaje de tipado estático que corre en la JVM, con la promesa de "escribir una vez, ejecutar en cualquier parte". Es habitual en banca, gobierno y grandes empresas, por su estabilidad de décadas y su gran ecosistema.

Java 8

Versión de Java de 2014 que sigue muy extendida en sistemas empresariales antiguos. Migrar a versiones más nuevas es una tarea típica de modernización.

JavaScript *JavaScript*

Único lenguaje que los navegadores ejecutan de forma nativa. Da interactividad a las páginas web y, con Node.js, también sirve para programar el servidor. Es la base de buena parte del desarrollo web.

JVM *Java Virtual Machine*

Máquina virtual de Java: el programa que ejecuta el código Java compilado. Permite que un mismo programa corra en distintos sistemas operativos. Otros lenguajes también corren sobre ella.

Lenguaje de programación *Programming language*

Conjunto de reglas y palabras para dar instrucciones a un computador. Cada uno tiene fortalezas distintas, y elegir el lenguaje es una decisión de arquitectura, no solo de gusto.

Mainframe *Mainframe*

Computador central de gran tamaño y alta confiabilidad usado por bancos y grandes empresas, donde suelen vivir sistemas en COBOL.

Node.js *Node.js*

Entorno que permite ejecutar JavaScript fuera del navegador, por ejemplo para construir servidores y herramientas de línea de comandos. Brilla en aplicaciones web con mucha interacción y APIs.

npm *Node Package Manager*

Gestor de paquetes de Node.js y repositorio público de librerías de JavaScript, el más grande que existe. Con npm install se agregan dependencias a un proyecto.

Object Pascal *Object Pascal*

Lenguaje compilado, de tipado estático y orientado a objetos, derivado de Pascal. Es el lenguaje de Delphi. Se lee casi como inglés, con palabras como begin y end en lugar de llaves. Brilla en aplicaciones de escritorio Windows con base de datos.

PHP *PHP*

Lenguaje interpretado pensado para la web, que corre en el servidor. Mueve una gran parte de los sitios existentes, incluidos muchos sistemas antiguos que aún funcionan.

PHP 5

Versión antigua de PHP, ya sin soporte. Muchos sitios y sistemas antiguos siguen usándola, con problemas de seguridad y compatibilidad que justifican su modernización.

pip *pip*

Instalador de paquetes de Python. Con pip install nombre se descarga una librería para usarla en tus programas.

Python *Python*



Lenguaje interpretado de sintaxis simple y legible. Domina la inteligencia artificial, la ciencia de datos y el Big Data, y también se usa mucho para scripts y automatización.

Sistema legado *Legacy system*

Sistema antiguo, a menudo sin documentación, que sigue siendo crítico para el negocio. Cambiarlo da miedo porque nadie está seguro de todas las reglas que esconde.

SQL *Structured Query Language*

Lenguaje para consultar y modificar datos en bases de datos relacionales. Con SELECT se leen datos, con INSERT, UPDATE y DELETE se cambian. Casi todo sistema que guarda datos lo usa.

TIOBE *TIOBE Index*

Ranking mensual de popularidad de lenguajes de programación. Sirve como referencia general de tendencias, pero popularidad no significa que sea la mejor opción para un proyecto concreto.

Tipado estático y dinámico *Static / Dynamic typing*

Con tipado estático, el tipo de cada dato se declara y se verifica antes de ejecutar (C#, Java). Con tipado dinámico se resuelve al correr (Python, JavaScript). El estático atrapa errores antes; el dinámico es más flexible.

TypeScript *TypeScript*

JavaScript con tipos opcionales, que se compila a JavaScript normal. Los tipos ayudan a detectar errores antes de ejecutar y a que los editores sugieran mejor. Muy usado en proyectos web grandes.

Visual Basic 6 *VB6*

Entorno y lenguaje de Microsoft de fines de los 90 para crear aplicaciones de escritorio Windows rápidamente. Hoy es un legado: ya no tiene soporte, pero muchos sistemas empresariales siguen funcionando con él.

TEMA 06

06 Delphi y DBISAM

25 términos

El sistema legado del caso: cómo leerlo sin conocerlo

AUTOINC

Tipo de campo que se numera solo con cada nuevo registro, usado como identificador. En PostgreSQL se reemplaza por una columna de identidad.

Base de datos embebida *Embedded database*

Base de datos que vive dentro de la aplicación, en archivos locales, sin servidor independiente. Es simple de instalar, pero difícil de compartir entre muchos usuarios o servicios.

BeforePost

Evento de Delphi que se dispara justo antes de guardar un registro. Es un lugar típico para validaciones y reglas de negocio ocultas.

Cliente-servidor *Client-server*

Modelo donde la base de datos corre como un servicio aparte al que varias aplicaciones se conectan por red. PostgreSQL funciona así. Permite más usuarios, seguridad y herramientas.

CURRENCY

Tipo de dato para dinero en DBISAM y Delphi. Al migrar a PostgreSQL y C# debe convertirse a numeric y decimal, nunca a float, porque los decimales binarios generan errores de redondeo.

Data module *Data module*

Contenedor sin pantalla que concentra la conexión a la base de datos y los componentes de datos de la aplicación, para que varios formularios los compartan.

DBISAM *DBISAM*

Motor de base de datos embebido para Delphi, de Elevate Software. Guarda las tablas en archivos que la aplicación abre directamente, sin un servidor aparte, y ofrece un dialecto propio de SQL.

Delphi *Delphi*

Entorno de desarrollo visual de Embarcadero basado en Object Pascal. Fue muy popular para aplicaciones de escritorio con base de datos: se arrastran componentes al formulario y se conectan a tablas en poco tiempo. Hoy hay pocos desarrolladores disponibles.

Delphi 7

Versión de Delphi de 2002 que muchas empresas siguen usando, y que sirve de referencia para el sistema legado del curso. Es una versión antigua, con pocas herramientas modernas.

.dfm *Delphi Form*

Archivo con la descripción visual de un formulario: qué componentes tiene, dónde están y qué eventos tienen asignados. Puede abrirse como texto para entender la pantalla.

.dpr *Delphi Project*

Archivo principal de un proyecto Delphi: lista las units y arranca la aplicación. Equivale al punto de entrada del programa.

Embarcadero *Embarcadero*

Empresa dueña actual de Delphi, que lo heredó de Borland. Publica la documentación oficial del lenguaje y de sus herramientas.

Evento *Event*

Código que se ejecuta cuando ocurre algo, como un clic o el guardado de un registro. En Delphi se asocian a un componente (OnClick, BeforePost) y es donde suele esconderse la lógica de negocio.

Formato dd/mm/yy

Fecha con año de dos dígitos. Genera ambigüedad de siglo (¿1925 o 2025?) y confusiones entre día y mes, por eso al migrar se transforma a un tipo de fecha real.

Formulario *Form*

Ventana de una aplicación Delphi. Se compone de un archivo .pas con el código y uno .dfm con el diseño visual de la pantalla.

MEMO

Tipo de campo para texto largo y de largo variable. Al migrar suele convertirse en text.

OnClick

Evento de Delphi que se dispara al hacer clic sobre un componente, como un botón. Su código a menudo mezcla pantalla, reglas de negocio y acceso a datos, lo que dificulta migrarlo.

RAD *Rapid Application Development*

Desarrollo rápido de aplicaciones: construir pantallas arrastrando componentes y conectándolos a datos, con poco código. Delphi es el ejemplo clásico.

TDataSource

Componente de Delphi que conecta un conjunto de datos (tabla o consulta) con los controles visuales que lo muestran, como una grilla o un campo de texto.

try..except

Estructura de Object Pascal para manejar errores: el código dentro de try se ejecuta y, si falla, se pasa a la sección except. Un except vacío esconde los errores y es una señal de alerta.

TTable y TQuery

Componentes de Delphi que representan una tabla (TTable) o el resultado de una consulta SQL (TQuery). Las versiones de DBISAM, como TDBISAMTable y TDBISAMQuery, hacen lo mismo con ese motor.

Turbo Pascal *Turbo Pascal*

Compilador de Pascal de Borland, muy popular en los años 80. Su creador, Anders Hejlsberg, diseñó después Delphi y más tarde C#, por eso Delphi y C# se parecen.

Unit *Unit*

Archivo con extensión .pas, la unidad básica de código de un proyecto Delphi. Tiene una sección interface (lo que muestra a otros) y una implementation (cómo lo hace). Se parece a una clase o módulo.

uses

Cláusula de Object Pascal que lista las units que una unidad necesita. Ayuda a ver las dependencias entre archivos de un proyecto.

VCL *Visual Component Library*

Biblioteca de componentes visuales de Delphi para Windows: botones, tablas, campos de texto. Cada componente tiene propiedades y eventos.

TEMA 07

07 .NET y C#

La plataforma donde se construye el portal web

26 términos

appsettings.json

Archivo de configuración de una aplicación ASP.NET Core. Guarda ajustes como la conexión a la base de datos; los valores secretos no deben quedar escritos aquí.

ASP.NET Core *ASP.NET Core*

Framework de .NET para construir aplicaciones web y APIs. Maneja las solicitudes HTTP, la configuración y la inyección de dependencias. Brilla en APIs empresariales, con buen rendimiento y soporte de herramientas.

async / await

Palabras de C# para operaciones que tardan, como llamar a una base de datos, sin bloquear el programa. await espera el resultado mientras libera el hilo para otras tareas.

Blazor *Blazor*

Framework de .NET para construir interfaces web con C# en vez de JavaScript. Es una opción al migrar formularios Delphi a la web.

Clase y propiedad *Class / Property*

Una clase es el molde que define datos y comportamiento de un objeto. Una propiedad es un dato de la clase, expuesto de forma controlada. Delphi también tiene propiedades, por eso la traducción es directa.

decimal

Tipo numérico de C# para valores exactos, como dinero. A diferencia de float y double, no acumula errores de redondeo en los decimales. Es el tipo correcto para montos.

dotnet build / test / run

Comandos de .NET: build compila el proyecto, test ejecuta las pruebas y run lo pone en marcha. Son los tres comandos básicos del día a día.

Entity Framework Core *EF Core*

ORM oficial de .NET: traduce entre objetos C# y tablas de la base de datos, y genera el SQL por ti. Acelera el desarrollo, aunque a veces conviene SQL directo para consultas complejas.

[Fact]

Atributo de xUnit que marca un método como una prueba simple, sin parámetros. El test pasa si no lanza excepciones ni falla ninguna aserción.

Interfaz *Interface*

Contrato que lista qué métodos debe tener una clase, sin decir cómo se implementan. Permite cambiar una implementación por otra, por ejemplo una falsa en las pruebas.

LINQ *Language Integrated Query*

Forma de C# de filtrar, ordenar y transformar colecciones con una sintaxis parecida a SQL. Se lee casi como una frase.

Middleware *Middleware*

Pieza que procesa cada solicitud web antes o después del endpoint, por ejemplo para autenticar, registrar o manejar errores. Se encadenan en un orden definido.

Minimal API *Minimal API*

Forma sencilla de definir endpoints en ASP.NET Core con pocas líneas, sin crear controladores completos. Útil para APIs pequeñas.

Namespace *Namespace*

Espacio de nombres: forma de agrupar y organizar clases en C# para evitar choques de nombres. Es el equivalente a un módulo o paquete.

.NET *.NET*

Plataforma de desarrollo de Microsoft, gratuita y multiplataforma, que incluye el runtime para ejecutar programas, bibliotecas y herramientas. C# es su lenguaje principal.

Npgsql *Npgsql*

Proveedor que permite a .NET conectarse a PostgreSQL. EF Core y otras librerías lo usan por debajo.

NuGet *NuGet*

Gestor de paquetes de .NET, equivalente a npm o pip. Con él se agregan librerías de terceros a un proyecto.

Nulabilidad *Nullability*

Característica de C# que marca qué valores pueden ser nulos y avisa al compilador si no se controla. Previene los errores de referencia nula, de los más comunes.

Program.cs

Archivo de arranque de una aplicación ASP.NET Core: configura los servicios, define las rutas y pone en marcha el servidor.

Proyecto (.csproj) *Project*

Archivo que describe un proyecto .NET: versión de la plataforma, paquetes que usa y tipo de salida. Cada proyecto compila a una biblioteca o a un programa.

Record *Record*

Tipo de C# pensado para datos inmutables, que se comparan por valor. Útil para transportar información sin lógica.

Runtime *Runtime*

Componente que ejecuta un programa ya compilado. Sin el runtime correcto instalado, el programa no corre.

SDK *Software Development Kit*

Conjunto de herramientas para desarrollar en una tecnología: compilador, bibliotecas y comandos. El SDK de .NET incluye el comando dotnet. Para programar necesitas el SDK; para solo ejecutar, basta el runtime.

Solución (.sln) *Solution*

Archivo que agrupa varios proyectos de .NET que forman una misma aplicación, por ejemplo la API y sus tests.

[Theory] y [MemberData]

Atributos de xUnit para pruebas parametrizadas: [Theory] marca que el test se repite con distintos datos, y [MemberData] indica de dónde salen. Ideal para correr muchos casos de un archivo.

xUnit *xUnit*

Framework de pruebas más usado en .NET. Los tests se marcan con [Fact] (prueba simple) o [Theory] (la misma prueba con varios datos) y se ejecutan con dotnet test.

TEMA 08

08 Datos y SQL

Dónde viven los datos y cómo se consultan

19 términos

Base de datos relacional *Relational database*

Base de datos que organiza la información en tablas relacionadas entre sí, y se consulta con SQL. PostgreSQL, MySQL y SQL Server son ejemplos.

Clave foránea *Foreign key*

Columna que apunta a la clave primaria de otra tabla y relaciona ambas. Por ejemplo, el cliente de un pedido. La base puede impedir referencias a datos que no existen.

Clave primaria *Primary key*

Columna que identifica cada fila de forma única en su tabla, como el número de pedido. No se repite ni queda vacía.

Consulta parametrizada *Parameterized query*

Consulta SQL donde los valores se envían aparte, como parámetros, en vez de pegarse dentro del texto. Es la defensa principal contra la inyección SQL.

COPY *COPY*

Comando de PostgreSQL para cargar o exportar grandes cantidades de datos desde y hacia archivos como CSV de forma muy rápida.

CSV *Comma-Separated Values*

Formato de texto para tablas, con una fila por línea y los valores separados por comas. Es simple y lo leen casi todas las herramientas, pero no guarda tipos.

DDL *Data Definition Language*

Instrucciones SQL que definen la estructura de la base, como CREATE TABLE. Un archivo de DDL permite recrear la base desde cero.

Esquema *Schema*

Descripción de la estructura de una base de datos: tablas, columnas, tipos y relaciones. También puede referirse a un espacio de nombres dentro de la base.

Índice *Index*

Estructura que acelera las búsquedas en una columna, como el índice de un libro. Hace las lecturas más rápidas, pero hace un poco más lentas las escrituras.

JOIN *JOIN*

Instrucción SQL que combina filas de dos tablas relacionadas, por ejemplo cada pedido con los datos de su cliente.

numeric y float

Tipos numéricos de PostgreSQL. numeric guarda valores exactos y es el correcto para dinero. float es aproximado y puede producir errores de redondeo, así que no debe usarse para montos.

PostgreSQL *PostgreSQL*



Base de datos relacional de código abierto, robusta y muy extendida. Se usa en aplicaciones web, análisis y sistemas críticos. Es el destino de la migración desde DBISAM.

psql *psql*

Cliente de línea de comandos de PostgreSQL. Permite conectarse a la base, ejecutar consultas y revisar tablas desde la terminal.

Row Level Security *RLS*

Función de algunas bases, como PostgreSQL y Supabase, que limita qué filas puede ver o cambiar cada usuario. Si se omite en una aplicación, los datos pueden quedar expuestos.

Supabase *Supabase*

Plataforma que ofrece base de datos PostgreSQL y servicios listos para aplicaciones web. Aparece en un caso de seguridad por aplicaciones generadas con IA que no configuraron RLS.

Tabla, fila y columna *Table / Row / Column*

Una tabla guarda un tipo de dato, como pedidos. Cada fila es un registro (un pedido) y cada columna un atributo (fecha, cliente, total).

Transacción *Transaction*

Grupo de operaciones que se ejecutan todas o ninguna. Si algo falla a mitad de camino, la base deshace lo hecho. Evita que queden datos a medias.

UTF-8 *UTF-8*

Codificación de texto que soporta tildes, eñes y símbolos de todos los idiomas. Una mala codificación hace que "ñ" aparezca como símbolos extraños.

Zona horaria *Time zone*

Región cuyo horario se usa como referencia. Guardar fechas sin considerarla causa errores sutiles, sobre todo con horarios de verano.

TEMA 09

09 Terminal, Git, GitHub y editores

38 términos

Las herramientas de todos los días del desarrollador

Bash *Bash*

Shell más usado en Linux y macOS. Permite ejecutar comandos, encadenarlos y escribir scripts. Muchos ejemplos de documentación están escritos para él.

Branch *Branch*

Rama: línea paralela de trabajo dentro de un repositorio. Permite desarrollar algo nuevo sin afectar la versión principal hasta que esté listo.

Clone *git clone*

Descargar una copia completa de un repositorio remoto, con su historia, a tu computador.

Code review *Code review*

Revisión de código: otra persona lee tus cambios antes de aceptarlos, busca errores y propone mejoras. Con IA, la herramienta comenta pero la decisión sigue siendo humana.

Commit *Commit*

Punto guardado en la historia de un proyecto, con los cambios hechos y un mensaje que los explica. Es la unidad básica del trabajo en Git.

Commit chico *Small commit*

Commit que contiene un único cambio pequeño y coherente. Es más fácil de revisar, de entender y de deshacer que un cambio enorme. Con IA conviene pedir una tarea a la vez.

Conflicto de merge *Merge conflict*

Situación en que Git no puede combinar automáticamente dos cambios porque tocan las mismas líneas. Hay que resolverlo a mano, eligiendo qué versión se queda.

Control de versiones *Version control*

Sistema que registra cada cambio de un proyecto, quién lo hizo y cuándo, y permite volver atrás o trabajar en paralelo. Git es el más usado.

Diff *Diff*

Comparación que muestra exactamente qué líneas se agregaron, cambiaron o borraron entre dos versiones. Leer el diff antes de hacer commit es el hábito clave al trabajar con IA.

Exit code *Exit code*

Número con que termina un comando: 0 significa que salió bien y cualquier otro indica un problema. Los scripts, los pipelines y los hooks lo usan para decidir si continuar o detenerse.

Fetch *git fetch*

Descargar los cambios del repositorio remoto sin integrarlos todavía en tu rama. Sirve para mirar qué hay de nuevo antes de decidir.

Fork *Fork*

Copia personal de un repositorio ajeno en tu cuenta, para proponer cambios sin tener permiso de escritura en el original.

Git *Git*

Sistema de control de versiones distribuido, de código abierto. Cada copia del proyecto guarda su historia completa. Es la red de seguridad de quien programa con IA, porque permite deshacer cambios.

GitHub *GitHub*

Plataforma para alojar repositorios Git y colaborar: pull requests, issues, revisión de código y automatización con Actions. Es el lugar donde ocurre buena parte del trabajo en equipo.

.gitignore

Archivo que lista lo que Git debe ignorar: archivos temporales, dependencias y, sobre todo, secretos como .env.

IDE *Integrated Development Environment*

Entorno de desarrollo integrado: aplicación que reúne editor de código, terminal, depurador y herramientas para ejecutar pruebas, como Rider o WebStorm.

Issue *Issue*

Ficha dentro de GitHub para reportar un problema, pedir una mejora o registrar una tarea. En un issue también se puede mencionar a @claude.

JetBrains *JetBrains*

Empresa que crea entornos de desarrollo como Rider (para .NET), WebStorm (para JavaScript) e IntelliJ IDEA (para Java). Ofrecen plugins para usar Claude Code dentro del editor.

Merge *Merge*

Unir los cambios de una rama con otra, normalmente al terminar una funcionalidad. Si dos ramas tocaron las mismas líneas, aparece un conflicto de merge.

PATH *PATH*

Variable de entorno con la lista de carpetas donde el sistema busca los programas que escribes en la terminal. Si un comando "no se reconoce", muchas veces falta agregarlo al PATH.

Pipe *Pipe*

Símbolo | que envía la salida de un comando como entrada del siguiente. Permite combinar comandos pequeños en una cadena útil.

PowerShell *PowerShell*

Shell y lenguaje de scripts de Microsoft, estándar en Windows. Sus comandos y su sintaxis son distintos de bash, por eso algunos ejemplos no funcionan igual en Windows.

Pull request *Pull request (PR)*

Solicitud para que tus cambios de una rama se integren en otra, con espacio para que otros los revisen y comenten antes del merge. Un PR chico y bien descrito se revisa mejor.

Push y pull *git push / git pull*

Push envía tus commits al repositorio remoto. Pull trae los cambios del remoto y los integra en tu copia. Son la forma de sincronizarse con el equipo.

Rama de respaldo *Backup branch*

Rama preparada de antemano con una solución o un estado conocido, para quien se atrasa o falla algo durante un taller.

Rama protegida *Protected branch*

Rama, como main, con reglas que impiden cambios directos: obliga a pasar por pull request, revisión o tests en verde antes de aceptar un cambio.

Rebase *Rebase*

Reescribir una rama para que parta desde un punto más reciente, dejando una historia lineal. Es potente pero delicado: no se hace sobre ramas que otras personas ya usan.

Remote y origin

Un remote es el repositorio al que apunta tu copia, normalmente alojado en un servicio como GitHub. origin es el nombre habitual del principal.

Repositorio *Repository*

Carpeta de un proyecto bajo control de Git, con todo su historial de cambios. Puede estar en tu computador y también en un servicio como GitHub.

Revert y reset

Formas de deshacer en Git. revert crea un commit nuevo que anula otro, sin borrar la historia; reset mueve la rama hacia atrás y puede descartar cambios, por lo que se usa con cuidado.

Rider *Rider*

IDE de JetBrains pensado para .NET y C#. Permite abrir una solución, ejecutar tests y trabajar con Claude Code desde su terminal integrada.

Shell *Shell*

Programa que interpreta los comandos que escribes en la terminal. Los más comunes son bash (Linux y macOS) y PowerShell (Windows); la sintaxis de cada uno cambia.

Stash *git stash*

Guardar temporalmente los cambios sin terminar para limpiar tu carpeta de trabajo, y recuperarlos después. Útil cuando debes cambiar de rama a mitad de algo.

stdout y stderr

Las dos salidas de un comando: stdout para el resultado normal y stderr para los errores. Pueden redirigirse a archivos de forma independiente.

Tag *Tag*

Etiqueta que marca un commit específico, normalmente una versión publicada como v1.0.

Terminal *Terminal*

Ventana de texto donde das órdenes al computador escribiendo comandos, en vez de hacer clic. Los agentes de código como Claude Code trabajan principalmente desde ella.

Variable de entorno *Environment variable*

Valor con nombre que el sistema o una herramienta guarda fuera del código, como una clave o una ruta. Se usa para configurar un programa sin escribir datos sensibles en el repositorio.

WebStorm *WebStorm*

IDE de JetBrains pensado para JavaScript, TypeScript y desarrollo web con Node.js.

TEMA 10

10 CI/CD y GitHub Actions

Automatizar las pruebas y la entrega

18 términos

Acción *Action*

Paquete reutilizable que se usa dentro de un workflow, por ejemplo para instalar una herramienta. Conviene fijar su versión para no recibir cambios inesperados.

Build *Build*

Proceso de convertir el código fuente en algo ejecutable. En un pipeline, es normalmente el primer paso: si no compila, no se sigue.

CI/CD *Continuous Integration / Delivery*

Prácticas para integrar cambios con frecuencia y probarlos automáticamente (integración continua), y para dejar el software listo para publicarse sin trabajo manual (entrega continua).

claude-code-action

Acción oficial de GitHub que permite usar Claude en workflows: responde cuando alguien lo menciona con @claude (modo interactivo) o ejecuta una instrucción automáticamente ante un evento. Propone cambios; una persona decide.

concurrency

Opción de un workflow que limita ejecuciones simultáneas del mismo tipo, por ejemplo cancelando una ejecución antigua cuando llega un cambio nuevo. Ayuda a controlar costos.

Despliegue *Deploy*

Publicar una versión del software en un entorno donde se usa, como pruebas o producción.

Entorno *Environment*

Lugar donde corre el software: desarrollo (tu computador), pruebas o staging, y producción (donde lo usan los clientes). Cada uno tiene su propia configuración.

GitHub Actions *GitHub Actions*

Sistema de automatización de GitHub: ejecuta workflows ante eventos como un push o un pull request. Se configura con archivos YAML dentro del repositorio.

GITHUB_TOKEN

Credencial temporal que GitHub entrega automáticamente a cada ejecución de un workflow, con permisos limitados al repositorio.

Job y step

Un job es un conjunto de pasos que corre en una misma máquina dentro de un workflow. Cada step es una acción o comando individual, como compilar o ejecutar los tests.

permissions

Sección de un workflow que declara qué puede hacer con el repositorio. La buena práctica es partir con lo mínimo, por ejemplo solo lectura de contenidos, y ampliar solo si hace falta.

Pipeline *Pipeline*

Secuencia automática de pasos que recorre un cambio: compilar, probar, analizar y desplegar. Si un paso falla, el cambio no avanza.

Pipeline en verde *Green pipeline*

Pipeline donde todos los pasos pasaron. Es la condición habitual para aceptar un pull request. En rojo significa que algo falló y hay que revisarlo.

Runner *Runner*

Máquina que ejecuta los jobs de un workflow. GitHub ofrece runners propios y también se pueden usar los de la organización.

Secret *Secret*

Valor sensible, como una API key, guardado de forma cifrada en la configuración del repositorio y entregado a un workflow cuando lo necesita. Nunca se escribe en el código ni en el YAML.

timeout-minutes

Límite de tiempo de un job. Evita que una ejecución colgada corra sin fin y gaste recursos.

Trigger *Trigger*

Evento que inicia un workflow, como push, pull_request o una programación horaria.

Workflow *Workflow*

Archivo YAML dentro de .github/workflows que define qué automatización correr, cuándo y con qué pasos.

TEMA 11

11 Contenedores y entorno

Ejecutar servicios sin instalarlos a mano

11 términos

Alpine *Alpine*

Distribución de Linux muy pequeña. Las imágenes con "alpine" en el nombre, como postgres:16-alpine, pesan menos y descargan más rápido.

Contenedor *Container*

Instancia en ejecución de una imagen: un entorno aislado y liviano donde corre un programa. Se puede crear, detener y eliminar sin afectar tu sistema.

Docker *Docker*



Herramienta que empaqueta una aplicación y todo lo que necesita en un contenedor, para que funcione igual en cualquier computador. Se usa para levantar bases de datos y servicios de prueba con un solo comando.

Docker Desktop

Aplicación de Docker para Windows y macOS que incluye todo lo necesario para ejecutar contenedores en un computador personal. Debe estar abierta para que los comandos funcionen.

docker run

Comando que crea e inicia un contenedor a partir de una imagen. Sus opciones definen el nombre, los puertos, las variables de entorno y los volúmenes.

.env

Archivo local con variables de entorno, como claves y contraseñas. Nunca debe subirse al repositorio ni dejarse dentro del proyecto si un agente de IA trabaja ahí.

host.docker.internal

Nombre especial que permite que un contenedor se conecte con servicios de tu propio computador. En Linux puede requerir una opción adicional para funcionar.

Imagen *Image*

Plantilla de solo lectura con un programa y sus dependencias, a partir de la cual se crean contenedores. Por ejemplo, la imagen oficial de PostgreSQL.

Playwright *Playwright*

Herramienta para controlar un navegador desde código: probar interfaces web de punta a punta, tomar capturas y generar PDF desde HTML. Funciona con Python, JavaScript y .NET.

Puerto *Port*

Número que identifica un servicio dentro de un computador. PostgreSQL usa por defecto el 5432. Al publicar un puerto con -p, conectas tu máquina con el contenedor; si está ocupado, se elige otro.

Volumen *Volume*

Espacio de almacenamiento que sobrevive al contenedor. Sin volumen, los datos de una base dentro de un contenedor se pierden al borrarlo.

TEMA 12

12 APIs, formatos y documentación

21 términos

Cómo conversan los sistemas y cómo se explica lo que hacen

API REST *REST API*

Estilo de API web donde cada recurso, como un pedido, tiene una dirección (URL) y se manipula con verbos HTTP. Es la forma más común de que un frontend o un sistema hable con un servidor.

Código de estado *Status code*

Número con que el servidor resume el resultado: 200 es éxito, 201 es creado, 400 es una solicitud inválida, 401 y 403 tienen que ver con permisos, 404 es no encontrado y 500 es un error del servidor.

Contrato *Contract*

Acuerdo escrito sobre cómo se comunican dos partes: qué se envía y qué se recibe. Si se respeta, cada lado puede cambiar por dentro sin romper al otro.

Controlador y servicio *Controller / Service*

En una API, el controlador recibe la solicitud y responde. El servicio contiene la lógica de negocio. Separarlos hace el código más fácil de probar y de cambiar.

Diagrama como código *Diagram as code*

Describir un diagrama con texto en vez de dibujarlo, para poder versionarlo, revisarlo y regenerarlo con facilidad.

Documentación técnica *Technical documentation*

Material que explica cómo funciona un sistema: README, diagramas, decisiones de arquitectura y contratos de API. La generada por IA debe verificarse contra el código real.

DTO *Data Transfer Object*

Objeto simple que solo transporta datos entre capas o hacia el cliente, sin lógica. Evita exponer la estructura interna de la base de datos.

Endpoint *Endpoint*

Dirección concreta de una API que realiza una acción, por ejemplo GET /pedidos para listar pedidos. Cada funcionalidad nueva suele agregar uno o más endpoints.

HTTP *HyperText Transfer Protocol*

Protocolo con el que el navegador y los servidores se comunican en la web: el cliente hace una solicitud y el servidor responde.

Indentación YAML *YAML indentation*

En YAML, los espacios al comienzo de cada línea definen la jerarquía. Se usan espacios, nunca tabulaciones, y la cantidad debe ser consistente.

JSON *JavaScript Object Notation*

Formato de texto para intercambiar datos, con pares clave-valor entre llaves. Lo usan casi todas las APIs y muchos archivos de configuración, como settings.json.

Markdown *Markdown*

Formato de texto simple con símbolos como # para títulos y ** para negrita. Es el estándar para README y para los archivos que leen los agentes de IA.

Mermaid *Mermaid*

Herramienta que dibuja diagramas a partir de texto, como flujos o relaciones entre módulos. Como los diagramas son texto, se versionan en Git y la IA puede generarlos desde el código.

OpenAPI *OpenAPI*

Formato estándar para describir una API REST: sus endpoints, parámetros y respuestas. Sirve como contrato entre equipos y permite generar documentación y código.

@page

Regla de hojas de estilo que define el tamaño y los márgenes de la página al imprimir o generar un PDF desde HTML.

Paginación *Pagination*

Dividir una lista grande en páginas para no devolver miles de resultados juntos. La API define el tamaño de página y cómo pedir la siguiente.

PDF headless *Headless PDF*

Generar un PDF desde HTML con un navegador que corre sin ventana, por ejemplo con Playwright o con Chrome en modo headless. Permite producir documentos con diseño de forma automática.

Plantilla *Template*

Documento o archivo base con la estructura ya definida, que se reutiliza cambiando solo el contenido. Mantiene el mismo diseño sin rehacerlo cada vez.

README *README*

Archivo de presentación de un proyecto: qué hace, cómo instalarlo y cómo usarlo. Es lo primero que lee quien llega al repositorio.

Verbo HTTP *HTTP method*

Acción que se pide sobre un recurso: GET lee, POST crea, PUT o PATCH modifican y DELETE elimina.

YAML *YAML*

Formato legible para configuración, que usa sangrías en vez de llaves. Los workflows de GitHub Actions están escritos en YAML. Un espacio mal puesto cambia el significado o lo rompe.

TEMA 13

13 Pruebas y calidad del código

28 términos

Cómo saber que el software hace lo que debe

Aserción *Assertion*

Línea de un test que compara el resultado real con el esperado. Si no coinciden, el test falla.

Caso borde *Edge case*

Situación poco común en los extremos de lo esperado, como una lista vacía, cero personas o una fecha invertida. Es donde suelen esconderse los errores y lo que la IA a menudo omite.

Cobertura de código *Code coverage*

Porcentaje del código que se ejecutó durante las pruebas. Indica qué no se probó, pero no si lo probado está bien: una cobertura de 100 % puede igual esconder errores.

Code smell *Code smell*

Señal en el código que sugiere un problema de diseño, aunque funcione: funciones larguísimas, duplicación, nombres como Proc2.

Código limpio *Clean code*

Código fácil de leer, entender y modificar: nombres que explican, funciones pequeñas y una sola responsabilidad por pieza.

Código muerto *Dead code*

Código que nunca se ejecuta, por ejemplo una regla activada por un flag que siempre es falso. Confunde a quien lee y debe eliminarse tras confirmarlo.

coverlet

Paquete de .NET que mide la cobertura de código al ejecutar las pruebas con dotnet test.

Deuda técnica *Technical debt*

Costo futuro de haber elegido una solución rápida en vez de una buena. Como una deuda, genera "intereses": cada cambio posterior cuesta más.

Fixture *Fixture*

Datos o estado inicial preparado para que una prueba parta siempre de las mismas condiciones.

Golden master *Golden master*

Conjunto de entradas y salidas congeladas del sistema original, usado como referencia. La versión nueva debe producir exactamente los mismos resultados, salvo los defectos que se corrijan a propósito.

Mock y stub

Piezas falsas que reemplazan una dependencia real durante una prueba, como un servicio externo. Permiten probar sin red, sin base de datos y de forma repetible.

Mutation testing *Mutation testing*

Técnica que cambia el código a propósito, introduciendo errores pequeños, para ver si las pruebas los detectan. Si el cambio no hace fallar ningún test, esos tests son débiles. Stryker.NET lo hace en .NET.

Número mágico *Magic number*

Valor escrito directamente en el código sin explicar qué significa, como un 0.19. Debe reemplazarse por una constante o un parámetro con nombre.

Pirámide de tests *Test pyramid*

Guía que recomienda tener muchos tests unitarios, menos de integración y pocos end-to-end, porque los de abajo son rápidos y baratos de mantener.

Prueba de software *Software testing*

Verificar que un programa se comporta como se espera, probándolo con casos concretos. Las pruebas automáticas se escriben una vez y se pueden ejecutar miles de veces.

Red de tests *Test safety net*

Conjunto de pruebas que te permite cambiar el código con confianza: si algo se rompe, se nota enseguida. Sin red de tests no se debe refactorizar.

Refactorización *Refactoring*

Mejorar la estructura del código sin cambiar su comportamiento: nombres claros, menos duplicación, funciones más cortas. Si el comportamiento cambió, ya no es un refactor.

Regresión *Regression*

Error que aparece en algo que antes funcionaba, normalmente tras un cambio. Los tests automáticos existen sobre todo para detectarlas a tiempo.

TDD *Test-Driven Development*

Desarrollo guiado por pruebas: primero se escribe un test que falla, luego el código mínimo para que pase y después se mejora el diseño. Con IA se usa a menudo al corregir bugs: reproducir, probar, corregir.

Test de caracterización *Characterization test*

Prueba que registra lo que un sistema hace hoy, aunque esté mal, antes de modificarlo. Sirve como red de seguridad al migrar o refactorizar código que no entiendes del todo.

Test de integración *Integration test*

Prueba que verifica que varias piezas funcionan juntas, por ejemplo tu código con una base de datos real. Son más lentos pero detectan errores que los unitarios no ven.

Test end-to-end *End-to-end (E2E) test*

Prueba que recorre el sistema completo como lo haría un usuario, por ejemplo abriendo el navegador y haciendo clic. Son los más realistas, pero también los más lentos y frágiles.

Test inestable *Flaky test*

Prueba que a veces pasa y a veces falla sin que el código cambie, por depender del tiempo, del azar o de la red. Quita confianza en todo el conjunto de pruebas.

Test parametrizado *Parameterized test*

Una misma prueba que se ejecuta con muchos conjuntos de datos. Es ideal para validar una regla con decenas de casos cargados desde un archivo.

Test que siempre pasa *Always-passing test*

Prueba que nunca puede fallar, por ejemplo la que compara un resultado consigo mismo. Da una falsa sensación de seguridad y es un clásico error al generar tests con IA.

Test unitario *Unit test*

Prueba automática de una pieza pequeña de código, como una función, de forma aislada. Son rápidos y forman la base de la pirámide de tests.

Valor límite *Boundary value*

Valor justo en el borde de una regla, como el monto exacto desde el cual se aplica un descuento. Probar un poco antes, justo en el límite y un poco después descubre muchos errores.

Variable global *Global variable*

Variable accesible desde cualquier parte del programa. Hace difícil saber quién la cambia y complica las pruebas.

TEMA 14

14 Arquitectura y migración

Cómo se organiza un sistema y cómo se moderniza

23 términos

Acoplamiento y cohesión *Coupling / Cohesion*

El acoplamiento mide cuánto depende una pieza de otras; conviene que sea bajo. La cohesión mide qué tan relacionado está lo que hay dentro de una pieza; conviene que sea alta.

Alcance *Scope*

Límite de lo que se construye y lo que no. Dejar escrito lo que queda fuera evita que el agente y el equipo inventen funcionalidades que nadie pidió.

Arquitectura de software *Software architecture*

Decisiones de fondo sobre cómo se organiza un sistema: sus partes, cómo se comunican y qué tecnologías usa. Son difíciles y caras de cambiar después.

Big bang *Big bang migration*

Migrar todo el sistema de una sola vez. Es riesgoso, porque si algo falla todo falla junto, y por eso suele evitarse.

Capas *Layers*

Forma de ordenar el código en niveles con responsabilidades separadas, como presentación, lógica de negocio y acceso a datos. Evita mezclar todo, como en un botón que calcula impuestos y consulta la base.

Convivencia y conciliación

Durante una migración, el sistema viejo y el nuevo funcionan a la vez (convivencia) y se comparan sus resultados para explicar cualquier diferencia (conciliación).

Dependencia *Dependency*

Cualquier cosa que una pieza de código necesita para funcionar: otra clase, una librería o un servicio externo.

Feature flag *Feature flag*

Interruptor en el código que activa o desactiva una funcionalidad sin volver a desplegar. Permite probar con pocos usuarios y apagar rápido si algo sale mal.

Función pura *Pure function*

Función que, con las mismas entradas, siempre devuelve el mismo resultado y no toca nada fuera de ella. Es fácil de aislar, probar y migrar.

Inversión de dependencias *Dependency inversion*

Principio de diseño: el código importante depende de interfaces y no de detalles concretos, como una base de datos específica. Permite cambiar los detalles sin tocar la lógica de negocio.

Inyección de dependencias *Dependency injection*

Técnica donde una clase recibe lo que necesita desde afuera en vez de crearlo ella misma. Facilita cambiar implementaciones y probar con piezas falsas. ASP.NET Core la incluye.

Mantener, refactorizar, modernizar o reemplazar

Las cuatro opciones ante un sistema legado. Mantener: funciona y casi no cambia. Refactorizar: mismo stack, mejor código. Modernizar: stack nuevo por partes. Reemplazar: producto o servicio nuevo.

Matriz de decisión *Decision matrix*

Tabla que compara opciones con criterios y pesos, como costo, riesgo e impacto, para decidir con datos y no por intuición.

Modernización *Modernization*

Llevar un sistema antiguo a tecnologías actuales manteniendo su valor para el negocio, por partes y con control de riesgos.

Módulo *Module*

Parte del sistema con una función clara y una interfaz definida. Un módulo bien separado se puede migrar o reemplazar de forma independiente.

Monolito *Monolith*

Aplicación construida como una sola pieza, con todo junto. Es simple de desplegar al comienzo, pero se vuelve difícil de cambiar cuando crece.

ORM *Object-Relational Mapper*

Herramienta que convierte entre objetos del lenguaje y filas de una base de datos, para no escribir todo el SQL a mano. Entity Framework Core es el de .NET.

Regla de negocio *Business rule*

Condición propia del negocio que el software debe respetar, como un descuento del 5 % sobre cierto monto. Son el verdadero valor de un sistema y deben conservarse al migrar.

Regla oculta *Hidden rule*

Regla de negocio escondida en el código sin documentar: dentro de eventos de pantallas, en consultas SQL o en números mágicos. Encontrarlas es el corazón de analizar un legado.

Repositorio (patrón) *Repository pattern*

Clase que concentra el acceso a datos de una entidad, como los pedidos. El resto del programa le pide datos sin saber cómo se obtienen.

Rollback *Rollback*

Volver a la versión anterior cuando una publicación sale mal. Todo plan de migración debe incluir cómo hacerlo.

Strangler fig *Strangler fig pattern*

Patrón de migración incremental: se reemplaza un módulo a la vez, el sistema viejo y el nuevo conviven y el tráfico pasa gradualmente al nuevo. Su nombre viene de una planta que crece alrededor de otra.

Ventana de corte *Cutover window*

Periodo planificado en que se apaga el sistema viejo y se activa el nuevo, normalmente en horarios de poco uso.

TEMA 15

15 Seguridad, datos y gobernanza

33 términos

Usar IA sin exponer secretos, datos ni responsabilidades

Allow y deny

Reglas de permiso: allow lista lo que el agente puede hacer sin preguntar y deny lo que tiene prohibido. Una regla deny aislada puede no bastar, por eso se combina con otras capas.

Auditoría *Audit*

Revisión sistemática de un sistema o proceso para encontrar problemas y riesgos, con evidencia de cada hallazgo.

Cadena de suministro *Supply chain attack*

Ataque que no vulnera tu código sino algo que usas, como una extensión del editor o una librería, para llegar a ti. Se reduce instalando pocos plugins y solo de fuentes confiables.

Consentimiento y finalidad *Consent / Purpose*

Principios de protección de datos: la persona debe saber y aceptar para qué se usan sus datos, y solo se usan para ese fin declarado.

CVE *Common Vulnerabilities and Exposures*

Identificador público único de una vulnerabilidad de seguridad conocida, como CVE-2025-48757. Permite que todos se refieran al mismo problema.

Datos personales *Personal data*

Información que identifica a una persona: nombre, RUT, correo, teléfono. Están regulados por ley y no deberían pegarse en una herramienta de IA sin autorización ni controles.

Defensa en profundidad *Defense in depth*

Combinar varias capas de protección para que, si una falla, otra detenga el problema: reglas de permiso, sandbox, hooks y, la más fuerte, no tener secretos dentro del proyecto.

Dependencia maliciosa *Malicious dependency*

Librería que contiene código dañino o que no existe de verdad: la IA a veces recomienda paquetes inventados que un atacante puede registrar. Se verifica que cada dependencia sea real y confiable.

Falso positivo *False positive*

Alerta que parece un problema, pero no lo es. Por eso cada hallazgo de una IA se valida leyendo el código antes de aceptarlo.

Hallazgo *Finding*

Problema o riesgo detectado en una revisión o auditoría, con su ubicación, su gravedad y una propuesta de corrección.

Hook guardián *Guard hook*

Hook que revisa cada acción del agente y la bloquea si rompe una regla, por ejemplo tocar archivos .env o .pem. Convierte una política escrita en un control que se cumple sola.

Inyección SQL *SQL injection*

Ataque donde alguien escribe código SQL en un campo de texto y lo ejecuta tu base de datos, porque la consulta se armó pegando textos. Se evita con consultas parametrizadas.

ISO 27001 *ISO/IEC 27001*

Norma internacional para gestionar la seguridad de la información en una organización. Una empresa certificada demuestra que sigue un sistema de gestión de seguridad.

Ley 21.719

Ley chilena que actualiza la protección de datos personales y refuerza los derechos de las personas y las obligaciones de las empresas.

Ley 25.326

Ley argentina de Protección de los Datos Personales, que regula el tratamiento de datos de las personas. Es la base legal de la privacidad en Argentina.

Ley 8968

Ley de Costa Rica sobre protección de la persona frente al tratamiento de sus datos personales.

LFPDPPP

Ley Federal de Protección de Datos Personales en Posesión de los Particulares de México. Regula cómo las empresas privadas deben tratar los datos de las personas.

Licencia de software *Software license*

Condiciones bajo las cuales se puede usar, copiar o modificar un software. Algunas permiten casi todo y otras imponen restricciones.

Licencia MIT *MIT License*

Licencia de código abierto muy permisiva: se puede usar, copiar y modificar, incluso en proyectos comerciales, conservando el aviso de autoría.

Minimización de datos *Data minimization*

Principio de recopilar y usar solo los datos estrictamente necesarios. Menos datos significa menos riesgo si algo se filtra.

Mínimo privilegio *Least privilege*

Dar solo los permisos que una tarea necesita y nada más. Si un agente solo debe leer, no debería poder escribir ni usar la red.

OWASP *OWASP*

Organización abierta que publica guías de seguridad web. Su lista Top 10 resume los riesgos más comunes en aplicaciones y es lectura básica para cualquier desarrollador.

Política ejecutable *Executable policy*

Regla que está aplicada en la configuración y no solo escrita en un documento. Si depende de que alguien "se acuerde", todavía es solo una sugerencia.

Prompt injection *Prompt injection*

Ataque donde un texto escondido en un archivo, página web o respuesta de un servidor le da órdenes al agente, como "ignora tus reglas". Todo lo que el agente lee es potencialmente no confiable.

Propiedad intelectual *Intellectual property*

Derechos sobre lo creado, como el código. Al usar IA conviene aclarar de quién es el código generado, evitar copiar código con licencias restrictivas y seguir la política de tu empleador. No es asesoría legal.

Retención de datos *Data retention*

Cuánto tiempo guarda un proveedor lo que le envías. Bajo términos comerciales, Claude no entrena con tu código y retiene los datos por un periodo limitado; además, tu computador guarda las sesiones en texto plano.

Sandbox *Sandbox*

Entorno aislado donde un programa corre con límites estrictos de archivos y red. Si el agente se equivoca o es engañado, el daño queda contenido.

Secreto expuesto *Exposed secret*

Clave, contraseña o token que quedó visible en el código, en el repositorio o en un log. Quien lo encuentre puede usarlo. Se guardan en gestores de secretos o variables, nunca en el código.

Seguridad de aplicaciones *Application security*

Prácticas para que un software resista ataques y no filtre datos: validar entradas, proteger secretos, mantener dependencias al día y limitar permisos.

Severidad *Severity*

Nivel de gravedad de un hallazgo, normalmente alta, media o baja. Ayuda a decidir qué corregir primero.

SOC 2 *SOC 2*

Informe de auditoría independiente que evalúa los controles de seguridad y privacidad de una empresa de servicios. Es una referencia común al elegir proveedores.

Transferencia internacional *International data transfer*

Enviar datos personales a otro país, por ejemplo al usar un servicio de IA en la nube. Muchas leyes exigen cuidados especiales.

XSS *Cross-Site Scripting*

Ataque donde se logra que la página de tu sitio ejecute un script ajeno en el navegador de otros usuarios, por ejemplo a través de un comentario sin filtrar.

TEMA 16

16 Métricas, productividad y negocio

18 términos

Medir el valor real de adoptar IA

Análisis de sensibilidad *Sensitivity analysis*

Ver cuánto cambia el resultado al modificar un supuesto, por ejemplo si el ahorro real fuese la mitad. Muestra qué tan frágil o sólido es el cálculo.

Business Unit *BU*

Unidad de negocio: una división de una empresa con sus propios equipos y objetivos. En el curso, el área de Desarrollo con varios equipos.

Defecto escapado *Escaped defect*

Error que no se detectó durante el desarrollo y llegó a producción. Son más caros de corregir y se usan para evaluar la calidad real.

DORA *DevOps Research and Assessment*

Programa de investigación que definió cuatro métricas de entrega de software: frecuencia de despliegue, tiempo de entrega de un cambio, tasa de fallas por cambio y tiempo de recuperación.

Escenario conservador y optimista

Dos cálculos del mismo ROI: uno con supuestos prudentes y otro con supuestos favorables. Mostrar ambos da un rango honesto en vez de una cifra única.

Frecuencia de despliegue *Deployment frequency*

Cada cuánto publica el equipo cambios a producción. Una frecuencia alta, con buena calidad, indica un proceso saludable.

Hito *Milestone*

Punto importante de un plan que marca un avance medible, como "todos los equipos con AGENTS.md".

Informe ejecutivo *Executive summary*

Documento de una página para quien decide: el problema, qué se hizo, los resultados con su incertidumbre, una recomendación y el siguiente paso.

Lead time *Lead time for changes*

Tiempo que pasa desde que un cambio está programado hasta que está en producción. Mientras menor, más ágil es el equipo.

Líneas aceptadas *Lines accepted*

Cantidad de líneas generadas por la IA que el equipo aceptó. Es una métrica de actividad, no de valor, y por sí sola puede engañar.

Métrica *Metric*

Número que sirve para seguir cómo va algo. Una buena métrica mide valor, como velocidad y calidad de entrega, y no solo actividad, como líneas de código.

Roadmap *Roadmap*

Hoja de ruta que ordena en el tiempo los hitos y las metas de un plan, por ejemplo a 6 y a 12 meses.

ROI *Return on Investment*

Retorno de la inversión: (beneficio − costo) ÷ costo. Responde si lo invertido valió la pena. Sin supuestos escritos y sustentados, el número no vale.

Supuesto *Assumption*

Dato que se asume cierto para poder calcular, como las horas ahorradas por tarea. En un ROI, cada supuesto debe quedar escrito con su fuente.

Tasa de aceptación *Acceptance rate*

Porcentaje de las sugerencias de la IA que el desarrollador acepta. Sirve como señal de adopción, pero sola no demuestra valor: lo aceptado puede borrarse después.

Tasa de fallas por cambio *Change failure rate*

Porcentaje de publicaciones que causan un problema y obligan a corregir o revertir.

Tiempo de recuperación *Time to restore*

Cuánto tarda el equipo en restablecer el servicio cuando algo falla en producción.

Usuarios activos *Active users*

Personas que usan una herramienta en un periodo. Mide adopción: cuántos la usan, no qué tan bien.